HashGuard Initial Requirements

Project Details

Project Title: HashGuard — A Python-Based File Integrity Monitoring System

Course: Software Construction and Development

Instructor: Engr. Muhammad Umer Haroon

Group Members

Name	Roll Number
Usman Dawood Afridi	47
Siaf Akbar	43
Mushaf Mashood	39

Document Status

This document contains the initial requirements for HashGuard. The requirements may be refined as development progresses.

Problem Statement

Files can be changed or deleted without the user noticing. Checking every file manually takes time, especially when there are many files in different folders. HashGuard will save a digital fingerprint, called a hash, for each file and compare it with the hash calculated during a later scan. This will help users find files that have been added, changed, or deleted. The user can then review these changes to decide whether they were expected.

Project Objectives

- Create a trusted baseline of file hashes for a selected folder.
- Detect files that have been added, modified, or deleted since the baseline was created.
- Clearly report files that could not be checked.
- Present scan results in a way that is easy to understand.
- Allow users to export scan results for later review.

Intended Users

HashGuard is intended for students, developers, and other users who want to check important files on their computers for unexpected changes.

- Students can check whether their project files have changed since they created a baseline.
- Developers can check source code and configuration files for unexpected changes.
- Other users can check selected documents and folders for added, modified, or deleted files.

Functional Requirements

- **FR-01 — Select a Folder:** The application shall allow the user to select a local folder to scan.
- **FR-02 — Scan Files:** The application shall scan files in the selected folder and its subfolders, applying the configured exclusions.
- **FR-03 — Calculate Hashes:** The application shall calculate a SHA-256 hash for each readable file included in the scan.
- **FR-04 — Create a Baseline:** The application shall allow the user to save a reference containing the selected folder's absolute path, baseline creation time, hash algorithm, and exclusion settings. For each successfully checked file, it shall store the path relative to the selected folder and its SHA-256 hash. Files that could not be included shall be recorded, and the baseline shall be marked as incomplete.
- **FR-05 — Load and Validate a Baseline:** The application shall allow the user to load a saved baseline for comparison with a later scan. It shall check that the baseline contains the required information and matches the selected folder. If the baseline is invalid or belongs to a different folder, the application shall explain the problem and stop the comparison.
- **FR-06 — Detect Changes:** The application shall compare the current scan with the baseline and identify added, modified, deleted, and unchanged files where sufficient information is available. If a file or location was not successfully checked during baseline creation or the current scan, the application shall report the affected comparison as unverified rather than assign an unsupported change status.
- **FR-07 — Report Verification Errors:** The application shall report files or folders that could not be checked. Files in inaccessible locations shall not automatically be classified as deleted.
- **FR-08 — Configure Exclusions:** The application shall allow the user to exclude selected folders or filename patterns from scanning. The exclusions shall be recorded with the baseline and applied during comparisons.
- **FR-09 — Display Results:** The application shall display file paths, their statuses, and a summary of the scan results.

- **FR-10 — Export Reports:** The application shall allow the user to export scan results as a CSV file.
- **FR-11 — Preserve the Baseline:** Running a comparison scan shall not automatically overwrite the saved baseline. Creating a new baseline shall require an explicit user action.

Non-Functional Requirements

- **NFR-01 — Usability:** The interface shall use clear labels and understandable messages. File statuses shall be shown using text, so users do not have to rely only on colors.
- **NFR-02 — Reliability:** A failure to read one file shall not stop the application from checking other accessible files. Scans with errors shall be clearly marked as incomplete.
- **NFR-03 — Memory Efficiency:** The application shall read files in chunks when calculating hashes rather than loading an entire file into memory.
- **NFR-04 — File Safety:** Scanning shall not modify or delete the contents of monitored files. Application data and reports shall be stored outside the monitored folder.
- **NFR-05 — Maintainability:** The code shall be organized into separate modules for scanning, hashing, comparison, storage, reporting, and the interface.
- **NFR-06 — Testability:** The core hashing and comparison functions shall be testable without opening the desktop interface.
- **NFR-07 — Compatibility:** The initial application shall be developed and tested on Windows using Python 3. The tested Python version shall be recorded in the project documentation.

Prioritized Features

Must Have

- Select and scan a local folder and its subfolders.
- Calculate SHA-256 hashes for readable files.
- Create and load a baseline.
- Detect added, modified, deleted, and unchanged files.
- Report unreadable files and incomplete scans.
- Apply exclusion rules consistently.
- Preserve the baseline unless the user explicitly creates a new one.
- Display understandable scan results.

Should Have

- Provide a desktop interface for the scanning workflow.
- Export scan results as a CSV report.
- Allow users to filter results by file status.

Could Have

- Store and display previous scan results.
- Support periodic scanning while the application is open.
- Preserve and select between multiple baseline versions.

The Must Have features will be developed first. Should Have features remain planned for the semester version but will follow the core functionality. Could Have features are optional and depend on the time available.

Initial Assumptions

- The user will create the baseline from files they consider trustworthy. HashGuard will not determine whether those files were already unsafe.
- The user will have permission to access the selected folder. Any files that remain inaccessible will be reported as unverified.
- The user will keep the baseline in a trusted location outside the monitored folder and protect it from unauthorized changes.
- Files are expected to remain reasonably stable during a scan. If the application detects that a file changed while being read, it will report that the file could not be reliably verified.
- The application will run locally on the user's computer. An internet connection will not be required for its core scanning and comparison features.

Project Constraints

- The application will be developed in Python.
- Windows will be the initial development and testing platform. Support for other operating systems is outside the initial scope.
- The project must be developed within the semester schedule and with the resources available to the group.
- Each scan will cover one selected local folder and its subfolders. Remote computers and cloud storage services are outside the initial scope.
- SHA-256 will be used consistently for file content hashing.
- Baselines and reports will be stored locally, outside the monitored folder.
- The initial version will use scans started by the user. Periodic scanning is an optional later improvement.
- The initial version will skip symbolic links and Windows junctions rather than follow them to other locations.

Known Limitations

- HashGuard will detect differences from a saved baseline, but it will not determine whether a change was authorized or malicious.
- The application will not prevent file changes, remove malware, or restore modified or deleted files.
- Changes made and then reversed between scans may go undetected.
- A renamed or moved file will initially be reported as a deletion at its original path and an addition at its new path.
- Files that cannot be read will remain unverified. An incomplete scan cannot confirm the integrity of every file in the folder.
- If someone can alter both the monitored files and the trusted baseline, they may be able to hide changes from the application.
- Scanning time will depend on the number of files, their sizes, and the computer's storage performance.
- The initial version will check file contents. Changes only to file permissions or other metadata will not be treated as content modifications.